R-T-C-C-O MASTER PROMPT

Corrected & Verified Version — Build an Agentic AI Trip Planning System

Purpose: This version preserves the original prompt's intent and requirements while resolving internal inconsistencies, tightening validation, defining provider-dependent behavior, and making the requested implementation more deterministic and testable.

Source basis: Uploaded 14-page prompt. The original defines the R-T-C-C-O role/task/context/constraints/output/development structure and requires a production-ready modular Python trip planner. ■filecite■turn0file0■L2-L15■

Verification Summary

- Trip arithmetic is consistent: 12 adults + 2 children = 14 travelers; 8 Nov 2026 through 15 Nov 2026 is 8 calendar days / 7 nights; 7 rooms × 2 occupants = 14 room capacity.
- The prompt now explicitly separates deterministic validation/calculation from LLM reasoning and from live-provider facts.
- Provider-dependent results must carry an explicit status instead of being inferred or fabricated.
- Planner and Executor responsibilities are separated, with a typed execution-plan contract and tool-registry requirement.
- RAG behavior is clarified: retrieved knowledge is evidence, not permission to invent missing policy facts; failed retrieval falls back safely.
- The final-output schema, verification statuses, error handling, tests, environment configuration, and acceptance criteria are made explicit.
- The prompt now requires the generated project to remain runnable without live travel APIs by returning provider-required/unverified states rather than fake results.

R — ROLE

You are a Senior Python Developer, Agentic AI Architect, RAG Engineer, and Travel Planning System Designer.

You are responsible for designing and implementing a production-ready, modular Agentic AI Trip Planner in Python.

Use clean architecture, modular code, Pydantic validation, specialized AI agents, tool calling, Planner/Executor architecture, ChromaDB-based RAG, deterministic business rules, strict output validation, and robust error handling.

The system must be designed so that real external travel APIs can be connected later without changing the core orchestration architecture.

Do not fabricate live travel facts. The application must clearly distinguish verified, unverified, provider-required, and knowledge-base-unavailable information.

T — TASK

Build a complete Agentic AI Trip Planning project that accepts structured trip requirements and produces a validated, detailed trip plan.

- Accept and validate user trip inputs using Pydantic.
- Create a structured execution plan using a Planner Agent.
- Execute the plan using an Executor.
- Use specialized agents for transport, hotels, food, weather, maps, itinerary, policy, and validation as required by the input.
- Use tools/APIs for external information instead of allowing the LLM to invent live information.
- Use ChromaDB for the travel-policy Knowledge Base.
- Implement semantic retrieval with keyword-search fallback.
- Use a RAG Agent to retrieve and summarize only relevant policy/instruction content.
- Validate the final response with Pydantic.
- Generate a structured final trip plan.
- Clearly identify information unavailable because a live provider/API is not connected.
- Keep the system modular, testable, observable, and extensible.

The project must run locally from Cursor with:

```
python app.py
```

C — CONTEXT

Trip Requirements

- Origin: Ahmedabad
- Destination: Munnar
- Adults: 12
- Children: 2
- Total travelers: 14
- Travel mode: Plane
- Start date: 8 November 2026
- End date: 15 November 2026
- Duration: 8 days / 7 nights
- Hotel category: 5 Star

- Room occupancy target: 2 persons per room
- Required rooms: 7
- Food preference: Jain Food
- Payment method: Credit Card
- Cancellation policy: Required

Preserve these requirements throughout the workflow. Do not silently change traveler counts, dates, rooms, food preference, payment method, or cancellation requirement.

Important transport rule: Because the requested destination is Munnar and the travel mode is Plane, the transport layer must support a flight-search step plus the required onward-ground-transfer consideration. If no live transfer provider is connected, mark transfer details as provider-required rather than inventing them.

Architecture Flow

```
User Input
    ↓
Pydantic Input Validation
    ↓
Planner Agent
    ↓
Typed Execution Plan
    ↓
Executor
    ↓
Specialized Agents
    ■■■ Knowledge/RAG
    ■■■ Flight/Transport
    ■■■ Hotel
    ■■■ Jain Food
    ■■■ Weather
    ■■■ Maps/Distance
    ■■■ Itinerary
    ■■■ Policy/Cancellation
    ↓
Tools / APIs / Knowledge Base
    ↓
Evidence + Agent Results
    ↓
Itinerary / Final Synthesis
    ↓
Pydantic Output Validation
    ↓
Final Trip Plan + Verification Status
```

Specialized Agents

- Planner Agent
- Knowledge/RAG Agent
- Transport/Flight Agent
- Hotel Agent
- Food/Jain Food Agent
- Weather Agent
- Maps/Distance Agent
- Itinerary Agent
- Policy/Cancellation Agent
- Final Output Validator

Follow the RAG approach defined in the source prompt: Knowledge Base → Chroma index → retrieval → retrieved content passed to the LLM → answer restricted to retrieved evidence → integrated into Planner/Executor workflow. ■filecite■turn0file0■L89-L95■

C — CONSTRAINTS

1. Input Validation

Use Pydantic models and deterministic Python validators. Invalid trip data must never enter the planning workflow.

- adults + children == total_travelers
- end_date >= start_date
- duration_days == (end_date - start_date).days + 1
- duration_nights == (end_date - start_date).days
- required_rooms >= 1
- room_occupancy >= 1
- required_rooms × room_occupancy >= total_travelers
- required fields are present and non-empty
- travel mode is an allowed enum value
- hotel category is present when hotel planning is requested
- food preference is present when food planning is requested
- payment method is present
- cancellation requirement is preserved
- dates use an explicit ISO/internal representation and are rendered consistently for users

For this supplied trip, validation must resolve to 14 travelers, 8 days, 7 nights, and capacity for 14 travelers across 7 rooms.

2. Agent Responsibilities

Do not create one giant agent that performs every task. Use specialized agents with narrow responsibilities.

Planner Agent: decides what needs to be done, which agent/tool performs it, execution order, dependencies, required inputs, and expected outputs.

Executor: executes only the approved typed plan, invokes tools through TOOL_REGISTRY, captures errors, and passes results/evidence to downstream steps.

Agents must not bypass deterministic validation or invent provider results.

3. Flight / Transport Information

Create a Flight Search Tool that accepts origin, destination/airport context, departure date, return date, and traveler count.

The tool must not invent prices, airline availability, flight numbers, schedules, booking references, or seat availability.

If no live flight provider is connected, return a structured provider-required result, for example:

```
{"status":"Provider Required","message":"Live flight provider data is required."}
```

The transport layer should also support onward transfer planning between the flight arrival point and Munnar. No live route/time/price may be fabricated.

4. Hotel Information

Create a Hotel Search Tool considering destination, check-in, check-out, hotel category, number of rooms, room occupancy, traveler count, and cancellation requirement.

Never invent hotel availability, room availability, prices, ratings, or cancellation terms.

If live hotel data is unavailable, return a structured Provider Required or Unverified result with a reason.

5. Jain Food

Create a dedicated Food/Jain Food Tool or Agent.

- Verified Jain food — supported by a provider/source that explicitly verifies the requirement.
- Vegetarian food — vegetarian status only; not equivalent to Jain.
- General restaurant information — restaurant information without Jain verification.

Never claim guaranteed Jain preparation unless the provider/source explicitly verifies it. The final output must preserve this distinction.

6. Weather

Create a Weather Tool accepting destination, start date, and end date.

Do not generate or imply live weather forecasts from model memory. Without a connected weather provider, return Provider Required.

7. Maps / Distance

Create a Maps/Distance Tool capable of distance, estimated travel time, and route information.

Do not invent live routing information. If no provider is connected, return Provider Required.

8. Knowledge Base / RAG

Create a travel-planning Knowledge Base using ChromaDB. The collection name must be:

```
trip_knowledge
```

Create these functions:

```
build_knowledge_base()
retrieve_docs(query)
keyword_search(query)
search_knowledge_base(query)
```

- Index documents idempotently; repeated indexing must not create duplicates.
- Use stable document IDs or content hashes for deduplication.
- If semantic retrieval fails, use keyword-search fallback.
- Retrieval functions must return a predictable list/result structure and must not crash the entire application.
- The RAG Agent may only state policy/instruction facts supported by retrieved documentation.
- If the requested information is absent from retrieved documentation, explicitly return: 'This information is not available in the knowledge base.'
- Do not use the LLM's general knowledge to fill missing policy facts.

9. Planner Agent

Create a structured execution-plan model. The Planner must dynamically select steps based on user input.

```
{
  "steps": [
    {
      "step": 1,
      "agent": "knowledge_agent",
      "task": "Retrieve relevant trip planning policies",
      "depends_on": []
    },
    {
      "step": 2,
      "agent": "transport_agent",
      "task": "Search flight and onward transport options",
      "depends_on": [1]
    },
    {
      "step": 3,
      "agent": "hotel_agent",
      "task": "Search 5-star hotel options",
      "depends_on": [1]
    },
```

```
    {
      "step": 4,
      "agent": "food_agent",
      "task": "Find verified Jain food options",
      "depends_on": [1]
    },
    {
      "step": 5,
      "agent": "weather_agent",
      "task": "Retrieve weather information",
      "depends_on": []
    },
    {
      "step": 6,
      "agent": "itinerary_agent",
      "task": "Create day-wise itinerary using verified/evidence-backed results",
      "depends_on": [2, 3, 4, 5]
    },
    {
      "step": 7,
      "agent": "policy_agent",
      "task": "Verify cancellation and payment requirements",
      "depends_on": [1, 3]
    },
    {
      "step": 8,
      "agent": "output_validator",
      "task": "Validate final trip plan",
      "depends_on": [6, 7]
    }
  ]
}
```

Required dynamic rules:

- If food preference is Jain Food, include the Food Agent.
- If cancellation policy is Required, include the Policy Agent.
- If travel mode is Plane, include the Transport/Flight Agent.
- If hotel planning is requested, include the Hotel Agent.
- If weather is requested, include the Weather Agent.
- If itinerary generation is requested, include the Itinerary Agent and final validation.

10. Tool Registry

Create a central TOOL_REGISTRY. The Executor must call tools only through this registry.

```
TOOL_REGISTRY = {
    "search_knowledge_base": search_knowledge_base,
    "search_flights": search_flights,
    "search_hotels": search_hotels,
    "search_food": search_food,
    "get_weather": get_weather,
    "get_distance": get_distance
}
```

Tool interfaces must return structured results with status, data, source/provider information when available, and error details when applicable.

11. Output Validation

Use Pydantic models for final output validation. At minimum create:

- FlightOption
- HotelOption
- FoodOption
- TransferOption

- DayPlan
- VerificationStatus
- TripPlan

The final output must contain:

- trip summary
- traveler count
- transport information
- hotel information
- day-wise itinerary
- Jain food information
- cancellation policy
- payment method
- important notes
- verification/provider status
- source/evidence information where applicable

12. No Hallucination

Never fabricate flight prices, hotel prices, hotel availability, restaurant verification, Jain food guarantees, cancellation policies, weather, distance, travel times, booking references, or other live provider facts.

Use explicit statuses:

```
Verified
Unverified
Provider Required
Not Available in Knowledge Base
Error
```

When live information is unavailable, say: 'Live provider data is required.' When policy information is absent from RAG retrieval, say: 'This information is not available in the knowledge base.'

13. Model Usage

Use a strong reasoning-capable LLM for Planner decisions, complex orchestration, and final synthesis.

Use deterministic Python/Pydantic logic for validation, arithmetic, date calculations, capacity checks, enums, schema enforcement, and status rules.

Use external tools/APIs for live flight data, hotel availability, weather, maps, restaurant information, and other provider-dependent facts.

Do not ask the LLM to generate live travel facts from memory.

14. Project Structure

```
trip_planner/
■
■■■ app.py
■■■ config.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■
■■■ models/
■   ■■■ __init__.py
■   ■■■ trip.py
■   ■■■ execution.py
■   ■■■ output.py
■
■■■ agents/
```

```
■   ■■■ __init__.py
■   ■■■ planner_agent.py
■   ■■■ executor.py
■   ■■■ transport_agent.py
■   ■■■ hotel_agent.py
■   ■■■ food_agent.py
■   ■■■ weather_agent.py
■   ■■■ maps_agent.py
■   ■■■ itinerary_agent.py
■   ■■■ policy_agent.py
■   ■■■ final_agent.py
■
■■■ tools/
■   ■■■ __init__.py
■   ■■■ registry.py
■   ■■■ travel.py
■   ■■■ food.py
■   ■■■ weather.py
■   ■■■ maps.py
■   ■■■ providers/
■       ■■■ __init__.py
■
■■■ rag/
■   ■■■ __init__.py
■   ■■■ documents.py
■   ■■■ indexer.py
■   ■■■ retriever.py
■
■■■ prompts/
■   ■■■ planner_prompt.py
■   ■■■ agent_prompts.py
■   ■■■ output_prompt.py
■
■■■ tests/
■   ■■■ test_models.py
■   ■■■ test_rag.py
■   ■■■ test_planner.py
■   ■■■ test_executor.py
■   ■■■ test_output.py
■
■■■ chroma_db/
```

O — OUTPUT

Generate the complete project step by step.

- For every file, provide the exact file path.
- Explain the purpose of the file in one or two sentences.
- Provide complete runnable code.
- Do not provide pseudocode.
- Do not leave TODO placeholders for core functionality.
- Keep imports consistent across the project.
- Ensure the project can run with: python app.py.
- Use environment variables for secrets/API keys; never hard-code credentials.
- Include useful error handling and logging without exposing secrets.
- Keep provider adapters replaceable so adding real APIs does not require rewriting the core orchestration.

1. Installation Instructions

Provide exact commands for creating a virtual environment, installing requirements, and activation on Windows PowerShell, Windows CMD, and macOS/Linux.

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
```

```
# Windows CMD
.venv\Scripts\activate.bat
# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
```

2. Environment Variables

Create .env.example containing required configuration names only. Include provider/API settings as optional configuration where live providers are not yet connected. Never place real secrets in source code.

3. Test Cases

- valid trip input
- invalid traveler count
- invalid dates
- duration mismatch
- invalid room capacity
- missing required fields
- knowledge-base indexing
- duplicate/idempotent indexing
- semantic retrieval
- keyword fallback
- retrieval failure handling
- Planner output schema
- dynamic Planner rules
- Executor workflow
- tool-registry invocation
- provider-required behavior
- final Pydantic output validation
- unknown knowledge-base question
- Jain-vs-vegetarian status handling

4. Expected Execution

```
[1] Validating trip input
[2] Building / verifying knowledge base
[3] Creating execution plan
[4] Validating execution plan
[5] Executing planner steps
[6] Searching transport
[7] Searching hotels
[8] Checking Jain food
[9] Checking weather
[10] Retrieving cancellation policy
[11] Generating itinerary
[12] Validating final output
[13] Final trip plan generated
```

If a live provider is not configured, the corresponding stage must complete safely and report Provider Required rather than failing the entire trip-planning workflow.

5. Final Trip Output

- Trip Summary

- Traveler Details
- Transportation
- Hotel / Stay
- Day 1 through Day 8
- Jain Food Plan
- Cancellation Policy
- Payment Information
- Important Notes
- Verification Status

Clearly mark live/provider-dependent information as Verified, Unverified, Provider Required, Not Available in Knowledge Base, or Error.

6. Development Method

Build the project incrementally and verify each stage before moving to the next:

- 1. Pydantic models
- 2. Knowledge Base
- 3. Chroma indexing
- 4. Retrieval
- 5. Keyword fallback
- 6. Knowledge Base tool
- 7. Specialized tools
- 8. Planner
- 9. Executor
- 10. Final Agent
- 11. Output validation
- 12. End-to-end application

After each stage, run the relevant tests and fix errors before continuing. Do not skip validation, error handling, schema enforcement, or provider-status handling.

Acceptance Criteria

- The supplied trip input validates successfully.
- The system rejects inconsistent traveler/date/room data.
- The Planner produces a typed, dependency-aware execution plan.
- The Executor invokes tools only through TOOL_REGISTRY.
- RAG retrieval is idempotent and has a keyword fallback.
- The system never fabricates unavailable live travel information.
- Jain food is not represented as verified unless the source verifies it.
- Cancellation requirements are preserved and explicitly reported.
- The final response passes Pydantic validation.
- The application runs with python app.py even when live providers are absent.
- Tests cover the major validation, RAG, orchestration, fallback, and output paths.

- The architecture is ready for real travel API/provider adapters without redesigning the core workflow.

Verification Notes — Changes Applied

- Consistency: Confirmed 12 adults + 2 children = 14 travelers; 7 rooms × 2 = 14 capacity; 8 Nov–15 Nov 2026 = 8 days / 7 nights.
- Architecture: Added explicit execution-plan and dependency concepts so Planner output is machine-validatable.
- Transport: Clarified that a flight to the Munnar destination requires onward transfer consideration and that such live details must be provider-backed.
- RAG: Added idempotent indexing guidance, stable IDs/content hashes, predictable return behavior, and explicit unknown-answer handling.
- Validation: Added duration and room-capacity rules and clarified that deterministic validation occurs before orchestration.
- Provider safety: Standardized Verified / Unverified / Provider Required / Not Available in Knowledge Base / Error statuses.
- Project structure: Added missing execution, maps, food, provider-adapter, and test modules needed to satisfy the stated requirements.
- Testing: Expanded the requested tests to cover dynamic planning, fallback, provider-required behavior, and Jain-food verification.
- Execution: Clarified that missing live APIs must not crash the application; they must produce structured provider-required results.
- Output: Added evidence/source handling and final acceptance criteria to make the generated implementation easier to verify.

Note: This verification is an internal consistency and implementation-readiness review of the uploaded prompt. It does not claim that live flights, hotels, weather, restaurant availability, prices, or cancellation terms were verified.